

# 1. Software Design Specification

## CLARA

|                         |                                             |
|-------------------------|---------------------------------------------|
| Document Control        |                                             |
| Author(s)               | Michael Gardner                             |
| Document Version        | 0.1.0                                       |
| Applies to CLARA        | ^1.0                                        |
| Status                  | Draft                                       |
| Last Updated            | 2026-04-26                                  |
| SPDX-License-Identifier | BSD-3-Clause                                |
| License File            | See the LICENSE file in the project root    |
| Copyright               | © 2026 Michael Gardner, A Bit of Help, Inc. |

|                        |                                    |
|------------------------|------------------------------------|
| Project Profile        |                                    |
| Variant                | library                            |
| Library Role           | utility                            |
| Application Role       | N/A                                |
| Assurance              | spark-targeted                     |
| Execution              | sequential                         |
| Parallelism            | none                               |
| Platform               | desktop, server, embedded          |
| Execution Environment  | linux, macos, windows, ada-runtime |
| Processor Architecture | amd64, arm64, arm32                |
| Deployment             | native                             |

# Table of Contents

|                                                  |   |
|--------------------------------------------------|---|
| 1. Software Design Specification .....           | 1 |
| 2. Introduction .....                            | 1 |
| 2.1. Purpose .....                               | 1 |
| 2.2. Scope .....                                 | 1 |
| 2.3. References .....                            | 1 |
| 3. Architectural Overview .....                  | 1 |
| 3.1. Architecture Style .....                    | 1 |
| 3.2. Package Hierarchy .....                     | 1 |
| 3.3. Design Pattern: Generic Instantiation ..... | 1 |
| 3.4. Integration with Functional Library .....   | 2 |
| 4. Package Structure .....                       | 2 |
| 4.1. Clara (Root Package) .....                  | 2 |
| 4.2. Clara.Types .....                           | 2 |
| 4.3. Clara.Errors .....                          | 2 |
| 4.4. Clara.Application (Generic) .....           | 3 |
| 5. Data Structures .....                         | 3 |
| 5.1. Internal Argument Registry .....            | 3 |
| 5.2. Parse State .....                           | 4 |
| 6. Component Design .....                        | 4 |
| 6.1. Parsing Algorithm .....                     | 4 |
| 6.2. Help Generation .....                       | 4 |
| 7. Design Patterns .....                         | 5 |
| 7.1. Parsing Backend: Ada.Command_Line .....     | 5 |
| 7.2. Error Handling Pattern .....                | 5 |
| 8. Performance and Memory Design .....           | 5 |
| 8.1. Zero-Overhead Abstractions .....            | 5 |
| 8.2. Memory Management .....                     | 5 |
| 9. SPARK Readiness Assessment .....              | 5 |
| 9.1. Module Assessment .....                     | 5 |
| 10. Future Extensibility .....                   | 6 |
| 10.1. Subcommand Support .....                   | 6 |
| 10.2. Extension Points .....                     | 6 |
| 11. Design Decisions .....                       | 6 |
| 12. Appendices .....                             | 7 |
| 12.1. Change History .....                       | 7 |

## 2. Introduction

### 2.1. Purpose

This Software Design Specification (SDS) documents the architectural and detailed design decisions for **CLARA** (Command Line Arguments for Reliable Applications), a type-safe command-line argument parsing library for Ada 2022.

### 2.2. Scope

This document covers:

- package architecture and hierarchy,
- generic instantiation design pattern,
- integration with the Functional library,
- parsing backend design,
- data structures, and
- key design decisions and rationale.

### 2.3. References

- Software Requirements Specification (SRS).
- Ada 2022 Reference Manual (ISO/IEC 8652:2023).
- Functional Library – Result/Option monads.

## 3. Architectural Overview

### 3.1. Architecture Style

**CLARA** is a flat utility library organized by module. It is not a layered DDD/Clean/Hexagonal project. The primary structural concern is the **generic hierarchy**: `Clara.Application` is the root generic, with `Flag`, `Option`, and `Positional` as child generics.

### 3.2. Package Hierarchy

Clara (root package)

```
├─ Clara.Version          -- Library version information
├─ Clara.Types            -- Core type definitions (bounded strings)
├─ Clara.Errors           -- Parse error types
└─ Clara.Application      -- Generic CLI application definition
    ├─ Clara.Application.Flag      -- Child generic for boolean flags
    ├─ Clara.Application.Option    -- Child generic for value options
    └─ Clara.Application.Positional -- Child generic for positional args
```

### 3.3. Design Pattern: Generic Instantiation

**CLARA** uses Ada generic instantiation rather than a builder pattern. This is consistent with the Functional library and provides compile-time type safety.

| Aspect              | Builder Pattern           | Generic Instantiation   |
|---------------------|---------------------------|-------------------------|
| Ada Idiom           | Borrowed from OOP         | Native Ada pattern      |
| Consistency         | Different from Functional | Same as Functional      |
| Error Detection     | Runtime errors            | Compile-time errors     |
| SPARK Compatibility | Poor (heap allocation)    | Good (no heap)          |
| Type Safety         | Runtime (string lookups)  | Compile-time (packages) |

### 3.4. Integration with Functional Library

| CLARA Operation     | Functional Type           | Purpose                    |
|---------------------|---------------------------|----------------------------|
| Parse               | Result[Unit, Parse_Error] | Parse success/failure.     |
| Option.Value        | Option[String]            | Optional argument value.   |
| "or" operator       | From Option               | Default values.            |
| And_Then, Map_Error | From Result               | Railway-oriented chaining. |

Usage example:

```
My_CLI.Parse
  .And_Then (Validate_Args'Access)
  .And_Then (Build_Config'Access)
  .Map_Error (To_User_Error'Access);
```

```
Output_Path : constant String := Output.Value or "./default.txt";
```

## 4. Package Structure

### 4.1. Clara (Root Package)

```
package Clara with Pure is
  Library_Name : constant String := "CLARA";
end Clara;
```

### 4.2. Clara.Types

Core type definitions with SPARK-compatible bounds:

```
package Clara.Types with Pure is
  Max_Short_Length : constant := 1;
  Max_Long_Length  : constant := 64;
  Max_Help_Length   : constant := 256;
  Max_Value_Length  : constant := 1024;

  subtype Short_String is String (1 .. Max_Short_Length);
  subtype Long_String  is String (1 .. Max_Long_Length);
  subtype Help_String  is String (1 .. Max_Help_Length);
  subtype Value_String is String (1 .. Max_Value_Length);
end Clara.Types;
```

### 4.3. Clara.Errors

Parse error types compatible with Functional.Result:

```
package Clara.Errors with Pure is
  type Error_Kind is
    (Unknown_Switch,
     Missing_Value,
     Invalid_Value,
     Duplicate_Switch,
     Help_Requested);

  type Parse_Error is record
    Kind      : Error_Kind;
    Message   : Bounded_String;
    Context   : Bounded_String;
```

```

    end record;
end Clara.Errors;

```

## 4.4. Clara.Application (Generic)

The main generic package for defining a CLI application:

```

generic
  Name      : String;
  Description : String;
package Clara.Application is
  function Parse return Parse_Result.Result;
  procedure Print_Help;

  generic
    Short : Character := ASCII.NUL;
    Long  : String;
    Help  : String;
  package Flag is
    function Is_Set return Boolean;
    function Count return Natural;
  end Flag;

  generic
    Short      : Character := ASCII.NUL;
    Long       : String;
    Value_Name : String;
    Help       : String;
    Required   : Boolean := False;
  package Option is
    function Has_Value return Boolean;
    function Value return String_Option.Option;
  end Option;

  generic
    Name      : String;
    Help      : String;
    Multiple  : Boolean := False;
  package Positional is
    function Values return String_Vector;
    function Is_Empty return Boolean;
  end Positional;
end Clara.Application;

```

## 5. Data Structures

### 5.1. Internal Argument Registry

During parsing, CLARA tracks registered arguments:

```

type Argument_Kind is (Flag_Kind, Option_Kind, Positional_Kind);

type Argument_Def is record
  Kind      : Argument_Kind;
  Short     : Character;
  Long      : Bounded_String;
  Help      : Bounded_String;
  Value_Name : Bounded_String;

```

```

    Required    : Boolean;
    Multiple    : Boolean;
end record;

```

## 5.2. Parse State

After parsing, each argument stores its result:

```

type Flag_State is record
    Is_Set : Boolean := False;
    Count  : Natural := 0;
end record;

type Option_State is record
    Has_Value : Boolean := False;
    Value      : Bounded_String;
end record;

type Positional_State is record
    Values : String_Vector;
end record;

```

## 6. Component Design

### 6.1. Parsing Algorithm

```

FOR i IN 1 .. Argument_Count LOOP
    arg := Argument(i)

    IF arg starts with "--" THEN
        IF arg contains "=" THEN
            (name, value) := split on "="
            Store option value
        ELSE
            Find flag/option with matching Long name
            IF option requiring value THEN
                value := next argument
            END IF
        END IF
    ELSIF arg starts with "-" THEN
        FOR each char in arg[2..] LOOP
            Find flag/option with matching Short character
        END LOOP
    ELSE
        Add to positional arguments
    END IF
END LOOP

```

### 6.2. Help Generation

```

Print: Name " - " Description
Print: "Usage: " Name " [OPTIONS] " positional_names
Print: "Options:"
    FOR each flag/option: print short, long, help
Print: "Arguments:"
    FOR each positional: print name, help

```

## 7. Design Patterns

### 7.1. Parsing Backend: Ada.Command\_Line

**Decision:** Use Ada.Command\_Line instead of GNAT.Command\_Line.

| Aspect               | GNAT.Command_Line               | Ada.Command_Line |
|----------------------|---------------------------------|------------------|
| SPARK                | Not compatible                  | Compatible       |
| Standard             | GNAT-specific                   | Standard Ada     |
| Standalone Libraries | State isolation issues on macOS | Works correctly  |
| State                | Mutable, elaboration-time       | Stateless        |

**Issue encountered:** GNAT.Command\_Line with Library\_Standalone use "standard" causes command-line state to be isolated in a separate elaboration context on macOS. Arguments are not visible to code inside standalone libraries.

**Solution:** Use Ada.Command\_Line.Argument\_Count and Ada.Command\_Line.Argument(N) directly. These are stateless calls that work correctly in any library configuration.

### 7.2. Error Handling Pattern

Parse returns Result[Unit, Parse\_Error], integrating with the Functional library's railway-oriented programming model. Consumers chain parse results with And\_Then and Map\_Error without manual error checking.

## 8. Performance and Memory Design

### 8.1. Zero-Overhead Abstractions

- Bounded strings for all internal storage (no heap).
- O(n) parsing complexity where n is the argument count.
- Static dispatch via generics (no tagged types).

### 8.2. Memory Management

- All data structures are bounded and stack-allocated.
- No controlled types in core packages.
- Suitable for embedded and memory-constrained environments.

## 9. SPARK Readiness Assessment

### 9.1. Module Assessment

| Package                  | SPARK | Notes                     |
|--------------------------|-------|---------------------------|
| Clara                    | On    | Root package, pure.       |
| Clara.Application (core) | On    | Generic parsing logic.    |
| Clara.Errors             | On    | Error type definitions.   |
| Clara.Types              | On    | Bounded type definitions. |
| Clara.Version            | On    | Constants only.           |

## 10. Future Extensibility

### 10.1. Subcommand Support

The generic hierarchy naturally accommodates future subcommand support:

```
package My_CLI is new Clara.Application ("git", "Version control");
package Commit is new My_CLI.Command ("commit", "Record changes");
package Message is new Commit.Option ('m', "message", "MSG", "...");
```

Command would become a child generic of Application, with Flag, Option, and Positional as child generics of Command.

### 10.2. Extension Points

| Extension          | Mechanism                             |
|--------------------|---------------------------------------|
| Custom validators  | Generic formal function parameter.    |
| Help formatters    | Abstract interface + implementations. |
| Shell completions  | Additional generic procedures.        |
| Value transformers | Generic formal function parameter.    |

## 11. Design Decisions

| ID     | Decision                                    | Rationale                                                               |
|--------|---------------------------------------------|-------------------------------------------------------------------------|
| DD-001 | Generic instantiation over builder pattern. | Compile-time type safety, SPARK compatible, consistent with Functional. |
| DD-002 | Ada.Command_Line over GNAT.Command_Line.    | Stateless, no isolation issues in standalone libraries.                 |
| DD-003 | Depend on Functional library.               | Reuse Result/Option, railway-oriented processing.                       |
| DD-004 | Child generics for Flag/Option/Positional.  | Type-safe access, compile-time error detection.                         |
| DD-005 | Bounded strings for SPARK compatibility.    | No heap allocation, embedded-safe.                                      |

## 12. Appendices

### 12.1. Change History

| Version | Date       | Author          | Changes                                                  |
|---------|------------|-----------------|----------------------------------------------------------|
| 0.1.0   | 2025-12-19 | Michael Gardner | Initial design from discussion. Migrate to Typst format. |